

WELFARE CONNECT

AI-Based Multilingual Chatbot for Welfare Scheme Awareness

PDF 02 — SOLUTION FRAMEWORK

Accessible Works on WhatsApp — no app install, no portal login, no digital literacy beyond messaging.	Multilingual Detects and responds in the user's own language — Hindi, Tamil, Bengali, Hinglish, and 7+ more.	Intelligent RAG + Gemini-powered eligibility matching from a curated knowledge base of 15+ schemes.
---	--	---

Document Type	Solution Framework
Project	Welfare Connect — AI Multilingual Chatbot
Architecture	FastAPI + LangChain + FAISS + Gemini Flash + Twilio + React
Interfaces	WhatsApp (Primary), SMS Fallback, React Dashboard
Report Date	13 June 2026

1 Solution Overview

Welfare Connect is a **Retrieval-Augmented Generation (RAG)** conversational chatbot designed as a zero-friction, multilingual welfare scheme discovery engine. It operates natively on WhatsApp — the most-used messaging platform in rural India — requiring no app installation, no portal account, and no prior knowledge of available schemes.

The system combines three novel capabilities into a single interface:

+ Conversational Eligibility Profiling

A structured 5-question interview (age, gender, occupation, income, state) builds a profile without requiring the user to know what schemes exist or understand eligibility criteria.

+ Semantic Scheme Matching via RAG

User profiles are matched against a FAISS-indexed knowledge base of welfare schemes using Gemini's embedding model. The top-2 semantically similar schemes are retrieved and fed to the LLM as grounding context, preventing hallucination.

+ Multilingual Response Generation

Gemini Flash generates the final recommendation in the detected language of the user's messages. Bhashini API provides certified translation where needed, with Gemini's native multilingual capacity as a reliable fallback.

2 System Architecture

Layer	Component	Technology	Role
Entry Point	WhatsApp / SMS	Twilio Sandbox	Receives user messages; sends TwiML responses
API Server	REST Endpoints	FastAPI + Uvicorn	/webhook (WhatsApp), /sms, /download/{phone}
Session State	In-memory dict	Python dict (MVP)	Maintains conversation state per phone number
Language Layer	Translator	BhashiniTranslator + Gemini fallback	Detects language; translates input/output
Interview Engine	Prompt + State Machine	Gemini Flash (gemini-2.0-flash)	Conducts 5-question profile interview
Knowledge Base	schemes_data.json	JSON (15+ schemes)	Structured data on eligibility, docs, benefits

Vector Index	FAISS Index	FAISS-CPU + Gemini Embeddings	Semantic search over scheme documents
Embedding Model	GoogleGenerativeAI	models/gemini-embedding-2	Converts scheme text to 768-dim vectors
LLM Generation	Gemini Flash	LangChain + Gemini	Generates scheme recommendations + checklist
PDF Checklist	HTML Render	Python markdown + FastAPI	Downloadable document checklist per user
Dashboard	React SPA	React 19 + Vite 8	Glassmorphism analytics and demo UI

End-to-End Data Flow

1	User sends WhatsApp message to Twilio Sandbox number
2	Twilio POSTs message body + phone to /webhook (FastAPI)
3	BhashiniTranslator detects language; translates to English if needed
4	Session lookup: NEW → start interview EXISTING → continue
5	Interview phase: Gemini Flash asks 1 question at a time (Age/Gender/Occ/Income/State)
6	When LLM emits PROFILE_COMPLETE: token → profile extraction complete
7	RAG retrieval: FAISS top-2 scheme docs fetched for user's profile context
8	Gemini Flash generates personalized recommendation + download link
9	Response translated back to user's detected language
10	Twiml response returned to Twilio → WhatsApp message delivered to user

3 Technology Stack — Deep Dive

FastAPI Backend Framework	Async Python framework with automatic OpenAPI docs. Handles concurrent WhatsApp webhook requests with sub-10ms overhead. Chosen over Flask for async support critical for Gemini API latency.
LangChain LLM Orchestration	Provides the RetrievalQA chain, prompt template management, and Gemini integration. Enables clean separation of retrieval logic from generation logic.
FAISS-CPU Vector Search	Facebook AI Similarity Search — battle-tested, in-memory ANN index. k=2 retrieval returns top-2 scheme matches per user query in <5ms even with 1000+ scheme docs. No external vector DB required for MVP.
Google Gemini Flash LLM + Embeddings	gemini-2.0-flash for generation (low latency, multilingual native). models/gemini-embedding-2 for 768-dim semantic embeddings. Both via Google AI Python SDK through LangChain adapters.
Twilio WhatsApp Sandbox Messaging Layer	Zero-infrastructure WhatsApp integration. Sandbox provides real WhatsApp number for testing. Production path: WhatsApp Business API (same codebase, different credentials). SMS fallback via /sms endpoint with 300-char hard cap.
BhashiniTranslator Language Layer	India's government-backed translation API supporting 22 scheduled languages. Script-based language detection (Devanagari, Tamil, Bengali) + Bhashini translate. Graceful fallback to Gemini's native multilingual if API key unavailable.
React 19 + Vite 8 Dashboard Frontend	Glassmorphism dark-mode SPA with animated SVG ring gauges, WhatsApp mock UI, 3-persona demo flows, pilot test results, and district-level ROI calculator. No backend required — pure client-side demo dashboard.

4 RAG Pipeline — Technical Detail

Indexing Phase (startup)

1. Load schemes_data.json (15+ scheme documents)
2. Split into LangChain Document objects
3. Embed each document via GoogleGenerativeAIEmbeddings (models/gemini-embedding-2) — 768-dim vectors
4. Build FAISS flat index in memory
5. Expose as retriever with k=2

Index is rebuilt on every server start (MVP).
Future: persist to disk with FAISS.write_index() and load from cache.

Query Phase (per message)

1. User profile string assembled from interview answers
2. Profile embedded via same Gemini embedding model
3. FAISS ANN search: retrieve top-2 scheme docs
4. Docs injected into Gemini Flash prompt as context
5. LLM generates grounded recommendations (no hallucination)

The LLM is instructed to only recommend schemes present in the retrieved context. This is a hard constraint in the system prompt.

Key Code Patterns

```
# FAISS Vector Store Construction
embeddings = GoogleGenerativeAIEmbeddings(model='models/gemini-embedding-2')
vector_store = FAISS.from_documents(documents, embeddings)
retriever = vector_store.as_retriever(search_kwargs={'k': 2})

# SMS Hard Cap – code-level guarantee, not just a suggestion
SMS_MAX_CHARS = 300

def sanitize_sms_reply(text: str) -> str:
    sanitized = _URL_PATTERN.sub('', text).strip()
    if len(sanitized) > SMS_MAX_CHARS:
        sanitized = sanitized[:SMS_MAX_CHARS-1].rstrip() + '...'
    return sanitized

# PROFILE_COMPLETE token detection
if 'PROFILE_COMPLETE:' in llm_response:
    profile = extract_profile(llm_response) # regex parse key-value pairs
    session[phone]['state'] = 'RAG_READY'
```

5 Conversation Design — 5-Step Interview Engine

The interview engine is a state-machine built into the Gemini Flash system prompt. It ensures structured data collection before invoking the RAG pipeline:

Step	Field	Format	Scheme Relevance
Q 1	Age	Numeric	Determines age-specific schemes (PM Matru Vandana, NSAP pension, youth schemes)
Q 2	Gender	M/F/Other	Targets women-specific schemes (Sukanya, PMUY, PM Matru Vandana)
Q 3	Occupation	Free text	Identifies farmer, vendor, laborer, student → PM-KISAN, SVANidhi, MNREGA
Q 4	Income / BPL Status	BPL/APL/amount	Determines Ayushman, PMGKAY, housing scheme eligibility
Q 5	State / Location	State name	Applies state-specific schemes + location-based eligibility
—	PROFILE_COMPLETE	Token	LLM emits this special token when all 5 fields are captured; triggers RAG

Design Principle: The LLM is prompted to ask **exactly ONE question per turn**. If a user's response contains multiple pieces of information (e.g. '45 years, farmer, BPL'), the model extracts all of them and advances the state machine accordingly. This ensures natural conversation while maintaining structured data collection.

6 Financial Model & Unit Economics

Welfare Connect is designed for extreme cost efficiency. The entire stack can serve thousands of users at a cost measured in paise per conversation:

Cost Component	Rate (est.)	Per Conversation (10 turns)	Notes
Gemini Flash API	~\$0.0000875/1K tokens	~Rs.0.04	Input+output; Gemini 2.0 Flash pricing
Gemini Embeddings	~\$0.000025/1K chars	~Rs.0.002	One-time at FAISS build; negligible per query

Twilio WhatsApp	~\$0.005/message	~Rs.0.40	Sandbox free; Production ~Rs.0.40/msg (outbound)
Bhashini API	Free (govt API)	Rs.0.00	MeitY-funded; no per-call charge
FastAPI / Uvicorn	Rs.0 (serverless)	Rs.0.00	Can run on free-tier Railway/Render for pilot
FAISS (in-memory)	Rs.0 (CPU only)	Rs.0.00	No vector DB subscription required
TOTAL (MVP pilot)	—	~Rs.0.44/conversation	~Rs.4.40 for a 10-turn full session

Pilot Scale (1,000 users)

1,000 users × 10 turns = 10,000 messages
 Total API cost: ~Rs.4,400
 Each user = 1 scheme discovered
 PM-KISAN: Rs.6,000/yr per new enrolment

ROI: 1,364x

Rs.60 Lakh in benefits / Rs.4,400 cost

District Scale (10,000 users)

10,000 farmers in one district
 20% awareness lift = 2,000 new enrolments
 PM-KISAN: 2,000 × Rs.6,000 = Rs.1.2 Crore/yr
 Total API cost: ~Rs.44,000

ROI: 2,727x

Rs.1.2 Crore in benefits / Rs.44,000 cost

7 Scalability Architecture

MVP (Current)

12 pilot users,
3 days

**Rs.600 total
infra cost**

Twilio Sandbox + FAISS in-memory + Gemini Flash

- + FastAPI (local/Railway)
- + FAISS in-memory
- + Twilio Sandbox
- + Gemini Flash (free tier)

Pilot (Phase 2)

500+ users, 2
districts

**~Rs.15,000/
mo**

Twilio Production + Redis Sessions + Bhashini Full Integration

- + Railway/Render cloud
- + Redis session store
- + WhatsApp Business API
- + Bhashini certified translation

Scale (Phase 3)

50,000 users,
state-wide

~Rs.1.5L/mo

Production WhatsApp Business API + FAISS disk persistence

- + Kubernetes / GCP Cloud Run
- + FAISS disk index cache
- + 100+ scheme knowledge base
- + IVR voice channel

National (Phase 4)

10M+ users
nationwide

**Revenue /
grant funded**

NIC partnership + Government API Integration

- + NIC data center co-hosting
- + DigiLocker + PFMS API integration
- + 22 Bhashini languages
- + Offline SMS channel

8 Integration Architecture Map

Service	Provider	Integration Type	Data Flow	Status
WhatsApp	Twilio Sandbox	Webhook (HTTP POST)	Inbound: user messages; Outbound: TwiML	Live (Sandbox)
SMS Fallback	Twilio SMS	Webhook (HTTP POST)	Inbound: SMS; Outbound: plain text < 300 chars	Live
LLM	Google Gemini Flash	REST API (LangChain)	Prompt + context in; Generated text out	Live
Embeddings	Gemini Embedding-2	REST API (LangChain)	Text in; 768-dim float vector out	Live
Translation	Bhashini (MeitY)	REST API	Source text + lang codes in; translated text out	Scaffolded
Vector DB	FAISS (in-memory)	Python library	Text in; document + score pairs out	Live
Knowledge Base	Local JSON	File read at startup	15+ scheme dicts; loaded into LangChain Documents	Live
Document Checklist	FastAPI /download	HTTP GET	Phone number in; HTML checklist page out	Live
Analytics Dashboard	React/Vite :5173	Browser	Static demo data; no backend dependency	Live

9 Key Design Decisions & Trade-offs

WhatsApp over custom app

WhatsApp has 535M+ Indian users. A custom app would require installation, onboarding, and maintenance — all barriers for rural users. WhatsApp requires only a number to join and is already in everyday use. Penetration is near-universal among smartphone owners.

Trade-off: Trade-off: Twilio sandbox has participant limits; production requires WhatsApp Business API approval.

RAG over fine-tuning

Fine-tuning an LLM on scheme data would require retraining every time a scheme is updated. RAG allows real-time knowledge base updates (add a JSON entry) without touching the model. This is critical for a welfare domain where schemes change annually.

Trade-off: Trade-off: RAG quality depends on embedding model quality; k=2 may miss relevant schemes for complex profiles.

In-memory FAISS over vector DB

For MVP and pilot scale (<10,000 schemes), in-memory FAISS is faster, cheaper, and simpler than Pinecone, Weaviate, or pgvector. No subscription cost. Load time is <2s for 1,000 docs.

Trade-off: Trade-off: Index lost on server restart; no persistent storage for growing knowledge base.

Gemini Flash over GPT-4

Gemini Flash provides native multilingual comprehension in Indian languages without explicit translation for many use cases. It is significantly cheaper and faster than GPT-4 Turbo, enabling sub-3s response times on 2G networks.

Trade-off: Trade-off: Occasional quality gaps on very low-resource languages; mitigated by Bhashini.